

Probeklausur

Aufgaben — 90 Minuten, 100 Punkte

100
PUNKTE

90
MINUTEN

4
AUFGABENBLÖCKE

INHALT IN KÜRZE

TEIL 0 — Hinweise zur Bearbeitung

SO ARBEITEST DU DAMIT

- 1 Unter Klausurbedingungen bearbeiten.** Nur die SQL-Befehlsübersicht als Hilfsmittel, Uhr stellen, Lösung erst danach aufschlagen.
- 2 Punkte als Zeitbudget lesen.** Rund 55 Sekunden pro Punkt. Aufgabe 3.1 ist mit 12 Punkten die teuerste Einzelaufgabe.
- 3 Ergebnisse hinschreiben.** Bei „Ergebnis ermitteln“ zählt die hingeschriebene Ergebnistabelle, nicht der Gedankengang.
- 4 Danach die Musterlösung durchgehen.** Dort steht zu jeder Aufgabe auch die Punkteverteilung und die typischen Fehler.

FARBCODE DIESES LERNZETTELS

MUSS **Muss sitzen**
Kam in Klausuren mehrfach dran — sicher beherrschen.

WICHTIG **Prüfungsrelevant**
Kommt vor, aber nicht in jeder Klausur.

BASIS **Grundlagen**
Verständnis-Unterbau, selten direkt gefragt.

 **Merksatz**
Kernaussage — genau so formulierbar in die Klausur schreiben.

 **Falle**
Typischer Fehler bzw. Punktabzug-Klassiker.

 **Beispiel**
Durchgerechnete Aufgabe oder Musterlösung.

 **Frage**
Selbsttest — erst antworten, dann weiterlesen.

 **Originalton**
Wörtliches Zitat aus Folien oder Klausur.

 **Ergänzt**
Aus Fachliteratur ergänzt, nicht wörtlich aus den Folien.

 **Tipp**
Eselsbrücke oder Vorgehensstrategie.

Selbst erstellte Probeklausur. Sie bildet die vom Dozenten in „Beispiele Klausuraufgaben.pdf“ dokumentierten vier Aufgabenblöcke und Aufgabentypen nach und verwendet die Chinook-Beispieldatenbank der Übungen. Keine Originalklausur.

Inhaltsverzeichnis

© TEIL 0 — Hinweise zur Bearbeitung 2

Auszug aus dem Relationenschema `chinook`	3
Punkteübersicht	3

BLOCK 1 — SQL (40 Punkte) 3

Aufgabe 1.1 — Tabelle anlegen	3
Aufgabe 1.2 — Integritätsbedingung nachträglich ändern ..	3
Aufgabe 1.3 — Kann eingefügt werden?	4
Aufgabe 1.4 — Ergebnis der Abfrage ermitteln	4
Aufgabe 1.5 — Multiple Choice	4
Aufgabe 1.6 — Abfragen formulieren	5

BLOCK 2 — Functions und Trigger (25 Punkte) 5

Aufgabe 2.1 — Ergebnis einer Function ermitteln	5
Aufgabe 2.2 — Function schreiben	6
Aufgabe 2.3 — Trigger und Trigger-Funktion	6

BLOCK 3 — JDBC (20 Punkte) 7

Aufgabe 3.1 — Programmrumpf ergänzen	7
Aufgabe 3.2 — SQL-Injection	7

BLOCK 4 — Datenbankkonzepte (15 Punkte) 8

Aufgabe 4.1 — ACID und BASE	8
Aufgabe 4.2 — Anomalien und Isolation Level	8
Aufgabe 4.3 — NoSQL	8

© TEIL 0

Hinweise zur Bearbeitung

Modul	Moderne Datenbank-Konzepte (Praktische Umsetzung + Theoretische Konzepte)
Dauer	90 Minuten
Erreichbare Punkte	100
Hilfsmittel	SQL-Befehlsübersicht (Auswahl), nicht kommentiert
DBMS	PostgreSQL 16, Schema <code>chinook</code>

☰ Diese Probeklausur ist **nicht** die Originalklausur. Sie bildet die vom Dozenten in `Beispiele Klausuraufgaben.pdf` dokumentierten **vier Aufgabenblöcke und Aufgabentypen** nach und verwendet dieselbe Beispieldatenbank. Punkteverteilung und Umfang sind an das dort beschriebene Format angelehnt.

⚠ Bearbeitungshinweise

- Bei Multiple-Choice-Aufgaben können **mehrere Antworten richtig** sein. Es gibt Punkte pro korrekt gesetztem bzw. korrekt nicht gesetztem Kreuz.
- Wo nach dem **Ergebnis einer Abfrage** gefragt wird, ist das Ergebnis **hinzuschreiben** — Spaltenüberschriften und Zeilen, in der richtigen Reihenfolge.
- SQL-Schlüsselwörter dürfen groß oder klein geschrieben werden. Semikolon am Ende nicht vergessen.

Auszug aus dem Relationenschema `chinook`

```
artist      (artist_id PK, name)
album       (album_id PK, title, artist_id FK -> artist)
track       (track_id PK, name, album_id FK -> album, media_type_id FK,
            genre_id FK -> genre, composer, milliseconds, bytes, unit_price)
genre       (genre_id PK, name)
customer    (customer_id PK, first_name, last_name, company, city, country,
            email, support_rep_id FK -> employee)
employee    (employee_id PK, last_name, first_name, title,
            reports_to FK -> employee, hire_date)
invoice     (invoice_id PK, customer_id FK -> customer, invoice_date,
            billing_country, total)
invoice_line (invoice_line_id PK, invoice_id FK -> invoice,
            track_id FK -> track, unit_price, quantity)
playlist    (playlist_id PK, name)
playlist_track (playlist_id FK, track_id FK) -- zusammengesetzter PK
```

Punkteübersicht

Block	Thema	Punkte
1	SQL — DDL, DML, DQL	40
2	Functions und Trigger	25
3	JDBC	20
4	Datenbankkonzepte	15
Σ		100

BLOCK 1 — SQL (40 Punkte)

Aufgabe 1.1 — Tabelle anlegen

8 P.

Schreiben Sie **einen** SQL-Befehl, der die Tabelle `album` im Schema `chinook` erzeugt.

Anforderungen:

- `album_id` ist der **Primärschlüssel** und soll von der Datenbank **automatisch** vergeben werden.
- `title` ist eine Zeichenkette variabler Länge (max. 160 Zeichen) und **darf nicht leer sein**.
- `artist_id` ist ein **Fremdschlüssel** auf `artist(artist_id)`. Ein Künstler darf **nicht gelöscht** werden, solange noch Alben zu ihm existieren.
- `release_year` ist eine ganze Zahl und muss — sofern angegeben — **zwischen 1900 und 2100** liegen.
- Die Kombination aus `title` und `artist_id` muss **eindeutig** sein.

Aufgabe 1.2 — Integritätsbedingung nachträglich ändern

4 P.

Die Tabelle `customer` existiert bereits.

- Schreiben Sie den SQL-Befehl, der nachträglich erzwingt, dass `email` **eindeutig** ist. (2 P.)
- Schreiben Sie den SQL-Befehl, der die Spalte `phone` (Typ `varchar(24)`) zur Tabelle `customer` hinzufügt. (2 P.)

Aufgabe 1.3 — Kann eingefügt werden?

4 P.

Gegeben ist die Tabelle aus Aufgabe 1.1. In `artist` existiert **kein** Datensatz mit `artist_id = 99`.

Entscheiden Sie für jede Anweisung, ob sie **erfolgreich** ist, und **begründen Sie** in einem Satz.

SQL

-- a)

```
INSERT INTO album (title, artist_id) VALUES ('Kind of Blue', 99);
```

-- b)

```
INSERT INTO album (title, artist_id, release_year) VALUES ('Nevermind', 1, 1890);
```

Aufgabe 1.4 — Ergebnis der Abfrage ermitteln

8 P.

Gegeben ist die Tabelle `invoice`:

invoice_id	customer_id	billing_country	total
1	10	Germany	8,91
2	11	Germany	13,86
3	12	Canada	1,98
4	10	Germany	5,94
5	13	Brazil	13,86
6	12	Canada	8,91
7	14	USA	0,99
8	13	Brazil	1,98

Ermitteln Sie das **Ergebnis** der folgenden Abfrage. **Das Ergebnis ist hinzuschreiben.**

SQL

```
SELECT billing_country,
       count(*) AS anzahl,
       sum(total) AS umsatz
FROM   invoice
GROUP BY billing_country
HAVING count(*) > 1
ORDER BY umsatz DESC;
```

Aufgabe 1.5 — Multiple Choice

6 P.

Kreuzen Sie **alle** zutreffenden Aussagen an. Es können mehrere Antworten richtig sein.

a) Welche Aussagen über einen **INDEX** treffen zu?

- ☐ Ein Index stellt sicher, dass alle Werte einer Spalte eindeutig sind.
- ☐ Ein Index beschleunigt Such- und Abfragevorgänge.
- ☐ Ein Index verlangsamt **INSERT**, **UPDATE** und **DELETE**.
- ☐ Ein Index erzwingt die referentielle Integrität zwischen zwei Tabellen.

b) Welche Aussagen über **VIEW**s treffen zu?

- ☐ Eine View speichert die Ergebnisdaten physisch in der Datenbank.
- ☐ Über eine View mit **JOIN** und **DISTINCT** kann in der Regel nicht eingefügt werden.
- ☐ Views können zur Rechtevergabe auf Spalten- und Zeilenebene genutzt werden.
- ☐ Eine View wird bei jedem Zugriff neu ausgewertet.

c) Welche Aussagen über **DELETE** und **TRUNCATE** treffen zu?

- ☐ **TRUNCATE** kann mit einer **WHERE**-Bedingung eingeschränkt werden.
- ☐ **DELETE** löst Trigger aus, **TRUNCATE** in der Standardeinstellung nicht.
- ☐ Beide Befehle löschen die Tabellenstruktur mit.
- ☐ **DELETE** ohne **WHERE** löscht alle Zeilen der Tabelle.

Aufgabe 1.6 — Abfragen formulieren

10 P.

Formulieren Sie für jede Anforderung **einen** SQL-Befehl.

a) Die Titel aller Alben des Künstlers mit dem Namen **'Metallica'**. (3 P.)

b) Für jedes Genre die Anzahl der Tracks und die durchschnittliche Spieldauer in **Sekunden**, gerundet auf eine Nachkommastelle — nur Genres mit **mindestens 10 Tracks**, absteigend nach Anzahl. (4 P.)

c) Alle Kunden, die **noch nie** eine Rechnung hatten. Nutzen Sie eine **Unterabfrage**. (3 P.)

BLOCK 2 — Functions und Trigger (25 Punkte)

Aufgabe 2.1 — Ergebnis einer Function ermitteln

5 P.

Gegeben ist die Funktion:

```
SQL
CREATE FUNCTION chinook.anzahl(min_p numeric, max_p numeric)
RETURNS bigint
LANGUAGE sql
AS $$
    SELECT count(*) FROM chinook.invoice
    WHERE total >= min_p AND total <= max_p;
$$;
```

Die Tabelle **invoice** ist die aus **Aufgabe 1.4**. Welches Ergebnis liefert:

SQL

```
SELECT count(*) AS gesamtanzahl,  
       chinook.anzahl(2, 9) AS im_bereich  
FROM   chinook.invoice;
```

Geben Sie die Ergebniszeile an und **begründen Sie**, warum `gesamtanzahl` nicht von der Funktion beeinflusst wird.

Aufgabe 2.2 — Function schreiben

8 P.

Schreiben Sie eine Funktion `chinook.umsatz_kunde(p_customer_id int)`, die den **Gesamtumsatz** eines Kunden als `numeric` zurückgibt.

- Hat der Kunde keine Rechnungen, soll die Funktion `0` zurückgeben, nicht `NULL`.
- Verwenden Sie `LANGUAGE plpgsql`.
- Existiert der Kunde in `customer` **nicht**, soll die Funktion mit einer aussagekräftigen **Exception** abbrechen.

Aufgabe 2.3 — Trigger und Trigger-Funktion

12 P.

Beim **Einfügen** eines neuen Genres in `chinook.genre` soll geprüft werden, ob `name` belegt ist.

- Ist `name` `NULL` **oder** eine leere Zeichenkette, soll er automatisch auf `'Untitled Genre'` gesetzt werden.
- Zusätzlich sollen führende und abschließende Leerzeichen entfernt werden.

a) Schreiben Sie die **Trigger-Funktion**. (7 P.)

b) Schreiben Sie den **Trigger**, der sie aufruft. (3 P.)

c) Begründen Sie, warum hier `BEFORE` und nicht `AFTER` verwendet werden muss. (2 P.)

BLOCK 3 — JDBC (20 Punkte)

Aufgabe 3.1 — Programmerrumpf ergänzen

12 P.

Gegeben ist der bekannte Programmerrumpf:

```
JAVA
public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse = "jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        Connection conn = DriverManager.getConnection(url, props);

        // START Code zur Lösung der Aufgaben a) und b)

        // ENDE Code zur Lösung der Aufgaben a) und b)
        conn.close();
    }
}
```

a) Geben Sie `last_name` und `first_name` aller Kunden aus dem Land aus, das in der Variablen `String land` steht — zeilenweise auf der Konsole. Verwenden Sie ein `PreparedStatement`. (7 P.)

b) Setzen Sie anschließend für den Kunden mit `customer_id = 5` die Spalte `company` auf `NULL` und geben Sie aus, **wie viele Zeilen** geändert wurden. (5 P.)

Aufgabe 3.2 — SQL-Injection

8 P.

Gegeben ist folgender Code:

```
JAVA
String sql = "SELECT * FROM customer WHERE last_name = '" + eingabe + "'";
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(sql);
```

a) Geben Sie eine konkrete Eingabe für `eingabe` an, mit der **alle** Kundendatensätze zurückgeliefert werden, und schreiben Sie das entstehende SQL hin. (3 P.)

b) Schreiben Sie den Code so um, dass der Angriff **nicht mehr möglich** ist. (3 P.)

c) Erklären Sie in einem Satz, **warum** Ihre Lösung schützt. (2 P.)

BLOCK 4 — Datenbankkonzepte (15 Punkte)

Aufgabe 4.1 — ACID und BASE

6 P.

- a) Nennen und erläutern Sie die **vier ACID-Eigenschaften** in je einem Satz. (4 P.)
- b) Wofür steht **BASE**, und in welchem Punkt unterscheidet es sich grundlegend von ACID? (2 P.)

Aufgabe 4.2 — Anomalien und Isolation Level

5 P.

Zwei Transaktionen laufen parallel:

```
T1: SELECT sum(total) FROM invoice;           -- (1)
T2: INSERT INTO invoice (...) VALUES (...); COMMIT;
T1: SELECT sum(total) FROM invoice;           -- (2)
```

- a) Wie heißt die Anomalie, wenn (1) und (2) unterschiedliche Summen liefern? (2 P.)
- b) Welches **Isolation Level** verhindert sie, und welches ist in PostgreSQL die **Voreinstellung**? (3 P.)

Aufgabe 4.3 — NoSQL

4 P.

- a) Nennen Sie die **vier Kategorien** von NoSQL-Datenbanken mit je einem Vertreter. (2 P.)
- b) Das **CAP-Theorem** besagt, dass von drei Eigenschaften nur zwei gleichzeitig garantiert werden können. Nennen Sie die drei und ordnen Sie ein klassisches RDBMS ein. (2 P.)